

Inigo Quilez :: articles :: filterable procedurals

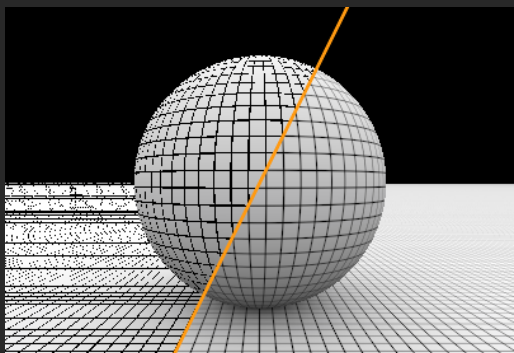


x filtered check

Intro

As described in [this article](#), checkerboard patterns can be filterable analytically, which makes them a great candidate for quality procedural texturing. Many other patterns accept simple analytic integrals and can therefore be filterable (antialiased) analytically. This article is a short collection of them. You can find live demos of these and some others here: <https://www.shadertoy.com/playlist/3KXR1>. Generalizations are pretty easy, for example getting a variety of dot/line patterns is straightforward, so I have only documented the basic ones for you to combine on your one:

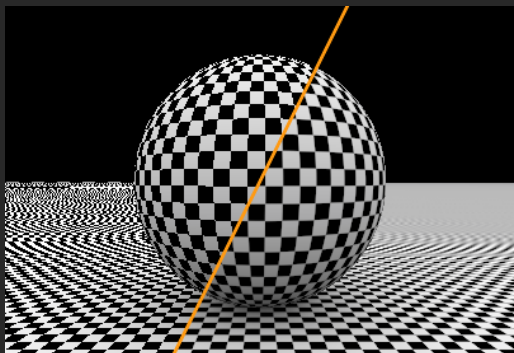
The List



Box filtered checkerboard

<https://www.shadertoy.com/view/XlcSz2>

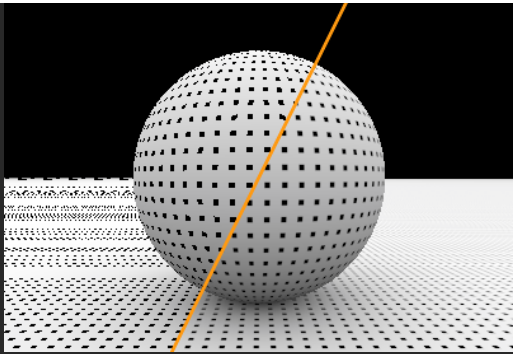
```
float filteredCheckers( in vec2 p, in vec2 dpdx, in vec2 dpdy )
{
    vec2 w = max(abs(dpdx), abs(dpdy));
    vec2 i = 2.0*(abs(fract((p-0.5*w)*0.5)-0.5)-
                abs(fract((p+0.5*w)*0.5)-0.5))/w;
    return 0.5 - 0.5*i.x*i.y;
}
```



Box filtered grid

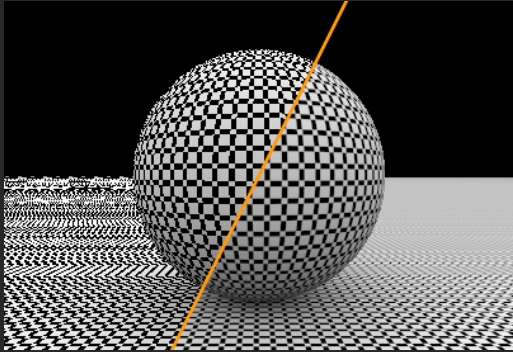
<https://www.shadertoy.com/view/XtBfzz>

```
float filteredGrid( in vec2 p, in vec2 dpdx, in vec2 dpdy )
{
    const float N = 10.0;
    vec2 w = max(abs(dpdx), abs(dpdy));
    vec2 a = p + 0.5*w;
    vec2 b = p - 0.5*w;
    vec2 i = (floor(a)+min(fract(a)*N,1.0)-
              floor(b)-min(fract(b)*N,1.0))/(N*w);
    return (1.0-i.x)*(1.0-i.y);
}
```



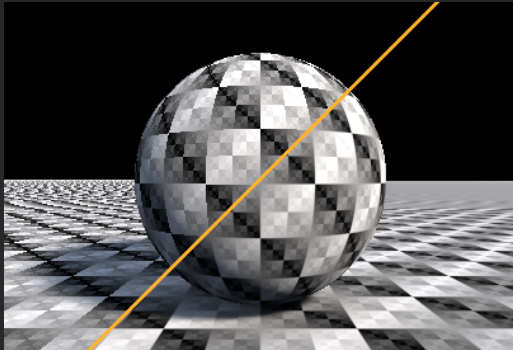
Box filtered squares

```
float filteredSquares( in vec2 p, in vec2 dpdx, in vec2 dpdy )
{
    const float N = 3.0;
    vec2 w = max(abs(dpdx), abs(dpdy));
    vec2 a = p + 0.5*w;
    vec2 b = p - 0.5*w;
    vec2 i = (floor(a)+min(fract(a)*N,1.0)-
              floor(b)-min(fract(b)*N,1.0))/(N*w);
    return 1.0-i.x*i.y;
}
```



Box filtered crosses

```
float filteredCrosses( in vec2 p, in vec2 dpdx, in vec2 dpdy )
{
    const float N = 3.0;
    vec2 w = max(abs(dpdx), abs(dpdy));
    vec2 a = p + 0.5*w;
    vec2 b = p - 0.5*w;
    vec2 i = (floor(a)+min(fract(a)*N,1.0)-
              floor(b)-min(fract(b)*N,1.0))/(N*w);
    return 1.0-i.x-i.y+2.0*i.x*i.y;
}
```



Box filtered XOR pattern

<https://www.shadertoy.com/view/tdBXRW>

```
float filteredXor( in vec2 p, in vec2 dpdx, in vec2 dpdy )
{
    float xor = 0.0;
    for( int i=0; i<8; i++ )
    {
        vec2 w = max(abs(dpdx), abs(dpdy)) + 0.01;
        vec2 f = 2.0*(abs(fract((p-0.5*w)/2.0)-0.5)-
                    abs(fract((p+0.5*w)/2.0)-0.5))/w;
        xor += 0.5 - 0.5*f.x*f.y;

        dpdx *= 0.5;
        dpdy *= 0.5;
        p *= 0.5;
        xor *= 0.5;
    }
    return xor;
}
```